KAIROS-6G: KPI-Aware Intelligent Reasoning for Operations and SUNRISE-6G

1st Paulito P. Palmes
AI for IT Automation
IBM
Dublin, Ireland
paulpalmes@ie.ibm.com

2nd Christos Verikoukis
ISI/ATH and University of Patras
Patras, Greece
verikoukis@athenarc.gr

3rd Kostas Ramantas
R+D Department
Iquadrat Informática
Barcelona, Spain
kramantas@iquadrat.com

***Abstract*—Federated 6G experimentation produces heterogeneous workflows, telemetry streams, model artifacts, and operational logs that are difficult to interpret during live trials and after experiment completion. This paper presents KAIROS-6G (KPI-Aware Intelligent Reasoning for Operations and SUNRISE-6G), an explainability layer for SUNRISE-6G that turns intent-driven MLOps into evidence-grounded operational reasoning. KAIROS-6G integrates Argo Workflows, *AutoMLPipeline*, *AutoAD*, MLflow, OpenTelemetry/Prometheus KPI streams, LLM reasoning, and Model Context Protocol (MCP) tool access in an operator-facing web command deck. Its core contribution is a closed loop from operator intent to workflow execution, KPI anomaly detection, experiment-memory capture, and human-readable explanation. *AutoAD* converts noisy KPI time series into detector-supported anomaly events, MLflow preserves run parameters and artifacts, and MCP gives the LLM scoped access to workflow, telemetry, and experiment evidence. The resulting explanations correlate KPI deviations with workflow stages, logs, detector settings, and model outcomes while preserving trust boundaries through read-only evidence gathering, redaction, source attribution, and approval-gated mutations. KAIROS-6G therefore supports sustainable SUNRISE-6G operations by making federated experiments more inspectable, repeatable, and reusable across testbeds and vertical use cases.**

***Index Terms*—LLM/MCP evidence gathering, KPI anomaly detection, explainable operations, time-series monitoring, MLOps**

I. Introduction

SUNRISE-6G [1] combines federated testbeds, service APIs, and native AI functions to support repeatable 6G experimentation across heterogeneous infrastructures. MLOps architectures emphasize workflow orchestration, reproducibility, metadata tracking, continuous monitoring, and feedback loops for automating and operating ML products [2]. Operators therefore need more than telemetry dashboards. They need to express operational intent, launch the corresponding machine-learning workflow, detect KPI deviations during execution, and retain enough evidence to explain whether a deviation is related to workload behavior, infrastructure conditions, or a workflow stage. KAIROS-6G addresses this need by connecting KPI-aware anomaly detection with explainable, intent-driven MLOps.

The companion AutoML-based workflow uses *AutoMLPipeline* [3], [4] and Argo Workflows [5] as the operational substrate for KAIROS-6G. Instead of hand-coding data preparation, model selection, monitoring, and reporting steps, reusable components are exposed through Model Context Protocol (MCP) tools [6] and executed as parameterized Argo Workflow templates. An operator can request anomaly detection over a KPI time series; the LLM-based agent [7] maps that intent to the appropriate MCP call, fills parameters such as input data, MLflow endpoint, run identifier, and anomaly-voting threshold, and lets Argo orchestrate the containerized tasks. Argo provides sequencing, parameterization, reuse, parallelism, and repeatability, while MLflow [8] and object storage capture logs, parameters, artifacts, and models for review.

KAIROS-6G uses *AutoMLPipeline* [3], [4] and its *AutoAD* components as the KPI anomaly-detection engine. *AutoAD* treats KPI streams, such as CPU, latency, throughput, packet loss, and service-level indicators, as tabular time-series inputs and applies an ensemble of unsupervised anomaly detectors. Its *AutoAnomalyDetection* workflow combines 16 anomaly detectors from pycaret and scikit-learn outlier-detection families [9], [10], including PCA, KNN, Histogram, Isolation Forest, Elliptic Envelope, One-Class SVM, and Local Outlier Factor. The configurable *votepercent* parameter controls the detector consensus required before KAIROS flags a KPI point as anomalous. Low thresholds expose weak or early signals; higher thresholds focus on stronger spikes or change points. The *AutoMLFlowAnomalyDetection* wrapper records thresholds, anomaly outputs, plots, and serialized artifacts in MLflow, making each detection run auditable rather than transient.

Together, these components form a closed loop for explainable operations. The operator states an intent, the agent selects and triggers the appropriate Argo workflow through MCP, *AutoAD* detects KPI anomalies, and MLflow stores the evidence. The KAIROS reasoning layer then correlates anomalous KPI intervals with workflow metadata, experiment context, logs, and MCP-accessible resources to summarize likely causes and next actions. This design keeps the workflow automated while letting operators reason at the level of experiment intent, KPI health, and evidence-backed explanation.

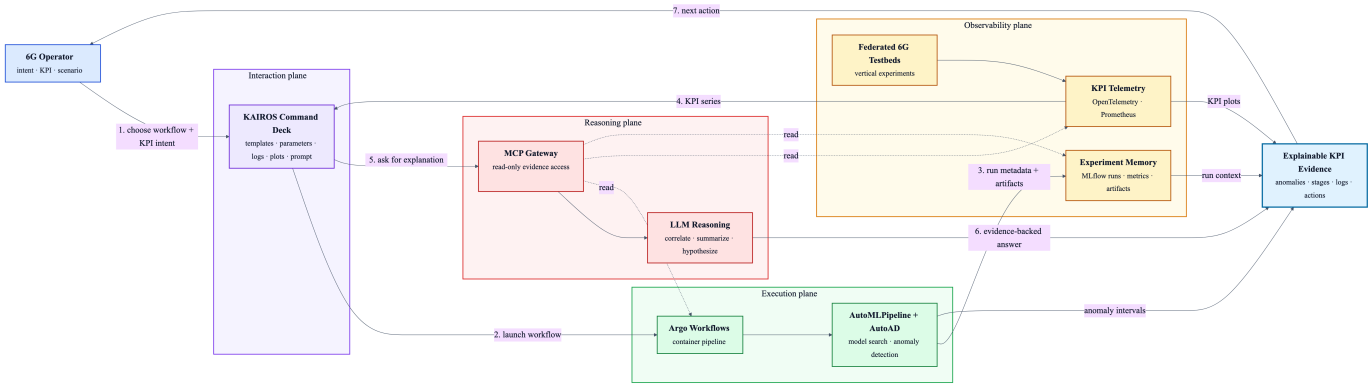


Fig. 1. KAIROS-6G architecture. The command deck links operator intent to Argo-based AutoML execution, KPI observability, *AutoAD* anomaly detection, MLflow experiment memory, and LLM/MCP-based explanation.

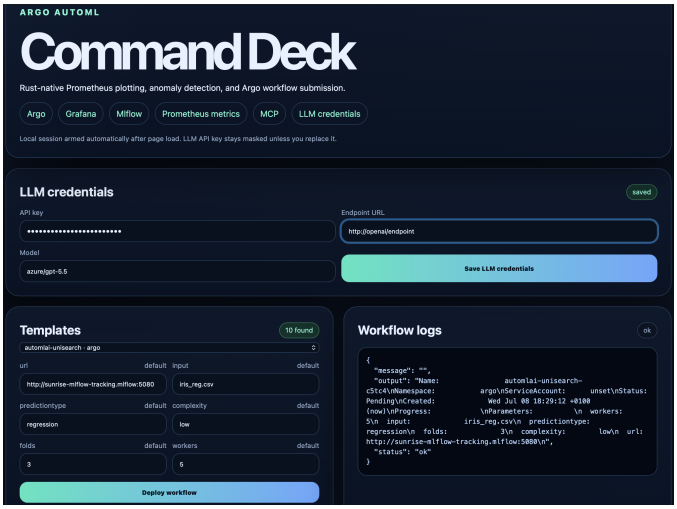


Fig. 2. KAIROS web command deck UI colocates LLM credentials, Argo template selection, workflow logs, KPI plot controls, Prometheus plots, anomaly streams, and the agent prompt in one operator workspace.

II. Architecture

Fig. 1 summarizes the architecture. KAIROS-6G is organized as an operator-facing command deck with four planes: interaction, execution, observability, and reasoning. The interaction plane is a browser command deck (Fig. 2) with panels for workflow templates, parameters, logs, Prometheus plots, anomaly streams, and agent prompts. Operators select an Argo workflow template and fill-up the parameter values necessary for the workflow to successfully run. Other panels retrieve workflow status, recent pod logs, CPU, memory, network, and latency plots. The prompt panel summarises the selected workflow, logs, MLflow result, and KPI plots. This layout keeps operational intent close to low-level evidence without requiring separate dashboards.

The execution plane provides reproducible MLOps. Argo Workflows runs the selected template as a containerized pipeline, while *AutoMLPipeline* supplies model-search and workflow components for regression, classification, and anomaly detection. For KPI analysis, Argo launches *AutoAD* with explicit parameters. This makes each intent replayable: the workflow template, parameters, name, stage history, logs, and artifacts can all be inspected. Argo also supports sequencing, parallel execution, retry, suspension, and resubmission, while KAIROS keeps mutating actions explicit and permission-gated.

The observability plane converts SUNRISE-6G activity into time-aligned evidence. OpenTelemetry exports infrastructure and service KPIs from federated testbeds, and Prometheus stores them for range queries. KPI anomaly detection is widely used for Web and service monitoring, including seasonal KPI streams [11]. KAIROS treats CPU, memory, network throughput, and latency as first-class metrics; the same pattern can support 6G service indicators such as packet loss, throughput, handover delay, slice quality, or vertical-application probes. MLflow stores parameters, metrics, tags, artifacts, and run metadata from *AutoMLPipeline* and *AutoAD* workflows, allowing KPI anomalies to be interpreted with the workflow and model context present at the time.

AutoAD converts KPI streams into events that the reasoning layer can inspect. The web interface supports quick interactive detection and full *AutoAD* mode. Quick mode runs a bounded detector ensemble for responsive previews, including Local Outlier Factor, Elliptic Envelope, PCA, and MCD. Full mode calls AutoAD's *AutoAnomalyDetection* workflow, which combines a total of 16 anomaly detectors from pycaret and scikit-learn outlier-detection families [9], [10]. Each detector votes on whether a KPI sample is anomalous, and *votepercent* sets the required consensus. The output is a plot with anomaly markers and structured metadata: metric name, query, time range, step size, detector mode, threshold, warnings, and anomalous timestamps.

The reasoning plane is evidence-first. Before the LLM generates an explanation, KAIROS builds a deterministic context from template metadata, workflow parameters, workflow status, logs, MLflow summaries, Prometheus plot data, and anomaly markers. MCP tools provide read-only access to Argo templates, workflow status, workflow logs, workflow

TABLE I
KAIROS-6G ARCHITECTURE COMPONENTS.

Component	Role	KAIROS contribution
Command deck	Presents templates, parameters, logs, plots, anomaly streams, and prompts.	Keeps operator intent close to evidence.
Argo Workflows	Runs reusable AutoML and *AutoAD* templates as containers.	Makes MLOps repeatable and auditable.
AutoMLPipeline	Builds model-search workflows for ML tasks.	Connects experiment data to automated ML lifecycle management.
AutoAD	Detects anomalous KPI samples using detector ensembles and *votepercent*.	Converts raw telemetry into explainable events.
OpenTelemetry and Prometheus	Collect and query infrastructure and service KPIs.	Provide timestamped evidence for correlation.
MLflow	Stores runs, parameters, metrics, tags, and artifacts.	Preserves experiment memory for comparison.
MCP tools	Expose scoped access to Argo, Prometheus, and MLflow evidence.	Enables controlled evidence gathering.
LLM layer	Summarizes evidence and proposes likely causes or next actions.	Turns telemetry and workflow state into operational hypotheses.

nodes, MLflow experiments, runs, metric history, and artifacts. Mutating actions such as submit, retry, stop, suspend, resume, delete, or resubmit remain gated by explicit permission and UI confirmation. Logs and MLflow data are redacted before model-context construction, and likely causes are labeled as hypotheses unless direct evidence supports causality.

Table I summarizes how each component contributes to the closed-loop architecture.

The KAIROS-6G control loop is:

- **Intent capture:** the operator selects a workflow, KPI, time range, anomaly marker, or asks a natural-language question.
- **Workflow automation:** KAIROS maps the selected intent to an Argo template and records the parameterized execution path.
- **KPI acquisition:** OpenTelemetry and Prometheus provide time-series evidence for infrastructure and service behavior.
- **Anomaly extraction:** *AutoAD* flags KPI samples using configurable detector consensus, producing auditable anomaly metadata.
- **Experiment memory:** MLflow links workflow outputs to parameters, metrics, and artifacts for comparison and replay.
- **Evidence-grounded explanation:** MCP retrieves scoped evidence, and the LLM summarizes correlations, confidence limits, and recommended next checks.

This architecture separates automation from explanation while connecting them through shared identifiers and timestamps. Argo and *AutoMLPipeline* automate the lifecycle; OpenTelemetry, Prometheus, *AutoAD*, and MLflow preserve evidence; MCP and the LLM make it understandable. As a result, a KPI deviation is detected and interpreted in the context of active experiments, workflow stages, detector settings, and model outcomes.



Fig. 3. KAIROS UI panels used in the agentic explanation loop: KPI plot controls define the metric, detector, time window, and vote threshold; the agent prompt panel turns selected evidence into an explanation request.

III. AGENTIC KPI EXPLAINABILITY

KAIROS-6G uses AI agents as evidence coordinators, not autonomous operators. The agent does not replace the workflow engine, telemetry stack, or anomaly detector. It reads their outputs through MCP tools, builds a compact evidence bundle, and explains KPI behavior in relation to workflow activity. This separation supports trust: Argo executes MLOps workflows, *AutoAD* detects anomalies, MLflow stores experiment evidence, Prometheus provides KPI time series, and the LLM reasons over collected facts.

Fig. 3 shows the corresponding UI controls for KPI evidence selection and agent prompting. The workflow begins when an operator asks why latency increased, why CPU usage spiked after deployment, or whether a workflow run caused a KPI deviation. KAIROS converts the question into a bounded investigation plan. The agent identifies the selected workflow, KPI, time range, anomaly marker, and detector settings, then uses MCP read-only tools to collect workflow status, node status, logs, MLflow summaries, metric history, artifacts, and

TABLE II
AGENTIC EVIDENCE WORKFLOW FOR KPI EXPLAINABILITY.

Step	Evidence source	Explainability outcome
Intent parsing	UI state: workflow, template, KPI, time range, anomaly marker.	Bounds the operator question.
Workflow inspection	Argo MCP: phase, nodes, times, retries, logs, arguments.	Shows activity near the KPI deviation.
KPI inspection	Prometheus and *AutoAD*: samples, anomaly times, detector mode, *votepercent*.	Separates variation from detector-supported anomalies.
Experiment lookup	MLflow MCP: parameters, metrics, tags, artifacts, plots.	Links KPI changes to model or detector context.
Evidence synthesis	LLM: time-aligned workflow, telemetry, anomaly, log, and MLflow facts.	Produces hypotheses, evidence, counter-evidence, and next checks.
Operator review	KAIROS UI: explanation, timestamps, plots, logs, artifacts.	Keeps interpretation auditable.

Prometheus plot metadata. This context contains timestamps, parameters, anomaly labels, workflow stages, and model outputs. The LLM uses it to generate an explanation with explicit evidence and confidence limits.

Table II maps each agent step to its evidence source and explainability outcome. MCP makes the agent useful without giving it unrestricted control. KAIROS exposes task-oriented tools to list workflow templates, inspect workflow status, retrieve logs and nodes, query MLflow experiments and runs, summarize or compare runs, retrieve metric history, and list artifacts. These tools return structured evidence rather than screenshots or free-form dashboard text. Their typed and scoped design lets the agent ask precise questions: which workflow nodes were active at a given time, which parameters were used, which artifact contains the anomaly plot, or whether two runs share a detector threshold. Mutating operations are separate and require explicit permission, keeping explainability read-only by default.

The agentic workflow follows three rules:

- **Evidence before reasoning:** the agent must first gather deterministic facts from Argo, Prometheus, *AutoAD*, and MLflow before generating a narrative.
- **Timestamp alignment:** a causal hypothesis must reference time overlap between KPI anomalies and workflow stages, logs, resource changes, or model activity.
- **Hypothesis labeling:** when evidence shows only correlation, the explanation is marked as a likely cause or candidate cause, not as a confirmed root cause.

AutoAD provides the event boundary for reasoning. Raw KPI plots are often noisy in federated 6G testbeds because background load, shared infrastructure, and vertical applications change over time. *AutoAD* reduces this noise by marking samples whose detector consensus exceeds *votepercent*. KAIROS gives the LLM anomaly timestamps, metric values, detector mode, and warnings rather than asking it to infer anomalies from raw plots. A latency anomaly can then be compared with Argo node start times, a CPU anomaly with model-search stages, and a memory anomaly with container logs or MLflow artifacts.

MLflow gives the agent memory across experiments. For one run, the agent can summarize parameters, metrics, tags, and artifacts. Across runs, it can compare detector thresholds, model choices, or output metrics to determine whether a KPI deviation is run-specific or recurring. This matters for SUNRISE-6G because an anomaly may reflect a repeated testbed pattern, vertical use-case profile, or model configuration rather than the current workflow alone.

The final KAIROS explanation is an evidence-backed triage note:

- **Observed anomaly:** metric, timestamp, value, detector setting, and severity cue.
- **Relevant workflow context:** active Argo stage, template parameters, node status, and log excerpts.
- **Experiment context:** MLflow run, metrics, artifacts, and detector or model configuration.
- **Likely explanation:** concise hypothesis with supporting and missing evidence.
- **Next action:** smallest operator check, such as inspect a pod log, compare a previous run, change *votepercent*, or rerun the workflow with the same parameters.

This design makes each explanation traceable to concrete components. Argo shows what was executed, Prometheus shows what changed, *AutoAD* marks where the change became anomalous, MLflow records experiment configuration and outputs, MCP provides controlled access to these facts, and the LLM turns them into an operational hypothesis.

IV. FUTURE

KAIROS-6G supports the long-term SUNRISE-6G goal of a sustainable pan-European 6G experimentation facility [12]. It turns experiment operations into repeatable, explainable, and low-touch workflows that remain useful after individual demonstrations finish.

A key sustainability challenge for federated 6G facilities is operational continuity. Testbeds, vertical applications, and component owners change over time, while experimenters need consistent ways to deploy workloads, inspect KPIs, and understand failures. KAIROS preserves each experiment as machine-readable evidence: Argo parameters and stages, logs, Prometheus KPI traces, *AutoAD* anomaly decisions, MLflow metrics, model artifacts, and LLM-generated explanations. This evidence can support future onboarding, troubleshooting, training, benchmarking, and cross-testbed comparison. Table III maps these operational capabilities to SUNRISE-6G sustainability goals.

Several extensions can strengthen this role. First, KAIROS should attach federation metadata to every evidence object, including testbed identity, vertical use case, experiment identifier, tenant scope, workflow name, and KPI namespace.

TABLE III
KAIROS VALUE FOR SUNRISE-6G SUSTAINABILITY.

Objective	KAIROS contribution	Sustainability value
Federated facility	Correlates KPI anomalies with workflow stages, logs, and metadata.	Reduces site-specific debugging and supports comparable operations.
Federation and CAMARA APIs	Exposes services, workflows, metrics, logs, and artifacts through MCP.	Replaces one-off scripts with reusable patterns.
Testbed onboarding	Captures templates, detector settings, metric queries, and explanations.	Helps new sites reuse proven operational recipes.
Vertical validation	Provides KPI-aware evidence for metaverse, robotics, NTN, and other trials.	Improves reproducibility and post-experiment reporting.
6G Library	Links component tests, KPI behavior, AutoML outputs, and explanations.	Makes component behavior easier to compare and maintain.
Intent-driven MLOps	Uses LLM/MCP agents for evidence gathering and explainability.	Lowers operational burden for low-touch management.
Trustworthy AI	Builds redacted, scoped evidence bundles with source attribution.	Supports explainability, privacy, and sovereignty.

This would help distinguish local site effects from cross-facility patterns. Second, KAIROS should move from single-run analysis to longitudinal knowledge extraction, where repeated anomalies, recurring workflow failures, and model-performance trends become reusable operational playbooks. Third, KAIROS can support SUNRISE-6G's open dataset commitment by exporting curated, redacted anomaly episodes containing the time window, KPI trace, detector settings, workflow context, and final explanation.

KAIROS can also move from reactive explanation to proactive assurance. A future version can monitor active workflows, detect KPI anomalies automatically, retrieve evidence through MCP, and notify the operator with a concise hypothesis and recommended next check. Write actions should remain approval-gated, but read-only investigation can be automated safely. This provides a practical path toward zero-touch service assurance without removing human accountability.

Finally, KAIROS can help make SUNRISE-6G sustainable as a community facility. Each experiment can leave operational memory: what was deployed, which KPIs changed, why the change mattered, what evidence supported the explanation, and what action was taken. Over time, this creates a shared knowledge base for experimenters, testbed owners, component providers, and vertical owners. The long-term value is not only better anomaly detection, but a maintainable federation where operational expertise is captured, reused, and improved across the SUNRISE-6G lifecycle.

ACKNOWLEDGMENT

This work was partly funded by the SUNRISE-6G project (Grant #: 101139257), co-funded by the European Union and Smart Networks and Services Joint Undertaking (SNS JU).

REFERENCES

[1] SUNRISE-6G Consortium, "SUNRISE-6G Project," https://sunrise6g.eu, 2024, accessed: 2026-07-08.

[2] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3262138

[3] P. Palmes. (2020, Aug.) Automlpipeline.jl: a package that makes it trivial to create complex ml pipeline structures using simple expressions. [Online]. Available: https://doi.org/10.5281/zenodo.3980593

[4] P. P. Palmes, A. Kishimoto, R. Marinescu, P. Ram, and E. Daly, "Designing machine learning pipeline toolkit for automl surrogate modeling optimization," *Proceedings of the JuliaCon Conferences*, vol. 1, no. 1, p. 129, 2023. [Online]. Available: https://doi.org/10.21105/jcon.00129

[5] Argo Project, "Argo Workflows," https://github.com/argoproj/argo-workflows, 2024, accessed: 2026-07-08.

[6] Model Context Protocol Contributors, "Model Context Protocol Documentation," https://modelcontextprotocol.io, 2024, accessed: 2026-07-08.

[7] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *ACM Trans. Intell. Syst. Technol.*, Jun. 2025, just Accepted. [Online]. Available: https://doi.org/10.1145/3744746

[8] MLflow Project, "MLflow," https://github.com/mlflow/mlflow, 2024, accessed: 2026-07-08.

[9] PyCaret Contributors, "PyCaret Anomaly Detection API," https://pycaret.readthedocs.io/en/latest/api/anomaly.html, 2024, accessed: 2026-07-08.

[10] scikit-learn Developers, "Novelty and Outlier Detection," https://scikit-learn.org/stable/modules/outlier_detection.html, 2024, accessed: 2026-07-08.

[11] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, D. Pei, Y. Feng, J. Chen, Z. Wang, and H. Qiao, "Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications," in *Proceedings of The Web Conference 2018*, 2018, pp. 187–196. [Online]. Available: https://doi.org/10.1145/3178876.3185996

[12] SUNRISE-6G Consortium, "SUNRISE-6G Objectives," https://sunrise6g.eu/objectives/, 2024, accessed: 2026-07-08.